

ViperGPT Reproduction: C1–C4 Prompt Specification Study

Professor-Facing Explanation Notes

MD Humayun

M.Tech CSE (Information Security & Privacy), SVNIT Surat

Supervisor: Dr. M. A. Zaveri

18 August 2026

Core story. C1 = released baseline; C2 = grounding contract; C3 = depth capability; C4 = depth capability *plus* routing rule.

Main finding. A capability is not sufficient. The specification must also define *when* that capability applies.

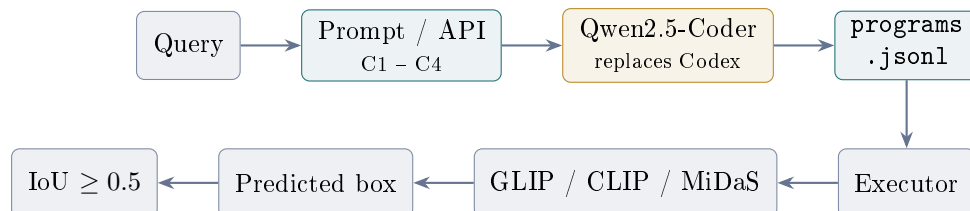
Contents

1	Pipeline flow	1
2	What is passed to the model?	1
3	C1 — released prompt / baseline	2
3.1	What is C1?	2
3.2	Why does this fail for grounding?	2
4	C2 — add the grounding contract	2
4.1	C2 results	3
5	C3 — add explicit depth capability	3
5.1	C3 result and problem	4
6	C4 — depth capability plus routing rule	4
6.1	C4 results	5
6.2	Robustness of the C4 result	5
7	The complete C1–C4 table	6
8	Spatial versus non-spatial — how to explain it	6
9	Why the C1–C4 design is a controlled experiment	6
10	What changed from the original ViperGPT paper?	7

11 What is the actual research contribution?	7
12 Summary	8
13 Questions and answers	8
14 Final	9

1 Pipeline flow

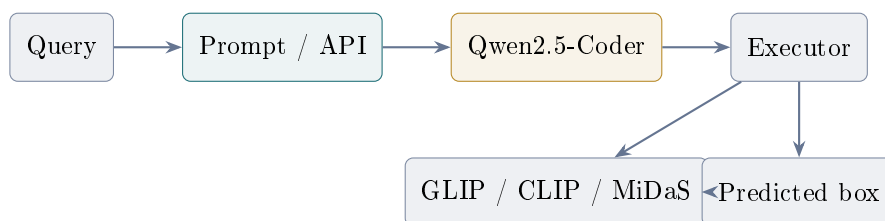
The original ViperGPT paradigm is kept: query $\rightarrow$ generated Python program $\rightarrow$ execution over vision modules $\rightarrow$ predicted region $\rightarrow$ IoU. The present-day generator is Qwen2.5-Coder because Codex is retired. Generation and execution are separated at a `programs.jsonl` boundary because the generator and the GLIP execution stack require different PyTorch environments.



“I am not changing the fundamental ViperGPT method. I keep the query–program–execute pipeline. My experiment studies whether the prompt/API specification is sufficient for grounding and depth reasoning. I therefore introduce four controlled conditions, C1 to C4, where each condition adds one specific intervention.”

2 What is passed to the model?

The generator is asked to write a Python program over an `ImagePatch`-style visual API. It never sees the image — only the query and the API specification.



The important research object is therefore not only the model. It is the **interface** between the language model and the visual API.

3 C1 — released prompt / baseline

3.1 What is C1?

C1 is the released task-agnostic prompt. It provides visual APIs and general Python instructions, but it does not explicitly say that a grounding program must return an `ImagePatch`.

A reported C1 generation for the query “*man on right*” is essentially:

```
def execute_command(image) -> str:
    image_patch = ImagePatch(image)
    man_patches = image_patch.find("man")
    if man_patches:
        man_patch = max(man_patches, key=lambda m: m.right)
        return man_patch.simple_query("Who is on the right?")
    return "No man found."
```

The program can be linguistically reasonable, yet it returns a string rather than a predicted region.

3.2 Why does this fail for grounding?

Grounding is evaluated by comparing a predicted box with a ground-truth box using IoU. A sentence has no predicted box, so its IoU is undefined:

Correct English answer $\nrightarrow$ valid grounding output.

In the reported RefCOCO baseline, **336 of 500** programs executed to completion and returned a string, while **0 of 500** returned a patch. The task-invalid return rate was 98.8% on RefCOCO and 98.2% on RefCOCO+.

Interpretation. C1’s 0% is not a crash. The evidence points to an output-contract failure: the program often produces text instead of an `ImagePatch`, so localisation is never reached.

Supporting evidence from the specification itself. The released prompt contains seven worked examples. Five declare a `str` return type, one declares `List[ImagePatch]`, and exactly one declares `ImagePatch` — and that one is buried inside the docstring of `compute_depth`, where a reader seeking the output contract has no reason to look. No sentence anywhere states that `execute_command` must return an `ImagePatch`.

4 C2 — add the grounding contract

$$\text{C2} = \text{C1} + \text{explicit grounding contract}$$

C2 adds a small prompt intervention: the function must return the single `ImagePatch` referred to by the query; it must not return a string, boolean or list; and spatial relations must not be dropped. Two worked grounding examples are also added. Nothing else in this comparison is changed.

For “*man on right*”, the desired pattern becomes:

```
def execute_command(image) -> ImagePatch:
    image_patch = ImagePatch(image)
    man_patches = image_patch.find("man")
    man_patches.sort(key=lambda p: p.horizontal_center)
    return man_patches[-1]
```

Now the chain is complete:

`ImagePatch` $\rightarrow$ predicted box $\rightarrow$ IoU.

4.1 C2 results

Metric	C1	C2
RefCOCO task-invalid return	98.8%	0.2%
RefCOCO+ task-invalid return	98.2%	0.0%
RefCOCO accuracy, IoU ≥ 0.5	0.00%	39.20%
RefCOCO+ accuracy, IoU ≥ 0.5	0.00%	31.80%
RefCOCO patch returns	0/500	443/500
RefCOCO string returns	336/500	1/500

Finding. C2 shows that a large part of the baseline failure is caused by an underspecified grounding return contract. The model did not suddenly become a better detector; the interface now clearly states what object type must be returned.

Two further pieces of evidence that this is the specification, not the model.

Scale. The same failure occurs at 1.5B, 7B and 32B parameters — 95.8%, 98.8% and 87.6% task-invalid respectively — and the same three sentences repair it at every size. Scaling the generator twentyfold buys about six accuracy points; stating the contract buys thirty-three to forty-one.

Format. Under C1 the model frequently did not define `execute_command` at all, emitting prose instead: 140/500 on RefCOCO and 212/500 on RefCOCO+, against zero under every later condition. The contract disciplines output format as well as return type.

5 C3 — add explicit depth capability

After C2 the output interface is fixed, but depth and 3D relational queries remain difficult — *closest, farthest, behind, in front of, between in depth*.

$$\text{C3} = \text{C2} + \text{explicit depth primitives}$$

C3 adds explicit operations:

Primitive	Purpose
<code>depth_order()</code>	order patches by relative depth
<code>is_behind()</code>	test a behind relation
<code>is_in_front_of()</code>	test a front relation
<code>is_between_3d()</code>	test a 3D between relation
<code>distance_3d()</code>	compare relative 3D separation

The semantic split introduced by C3 is:

$$\begin{aligned} \textit{left} / \textit{right} / \textit{above} / \textit{below} &\rightarrow \text{2D image coordinates} \\ \textit{closest} / \textit{farthest} / \textit{behind} / \textit{in front} &\rightarrow \text{depth primitives} \end{aligned}$$

The original API already had `compute_depth()`, but it exposed only a scalar. The reproduction observed **29** `compute_depth` calls against **547** `find` calls. C3 therefore gives the generator a direct vocabulary for depth relations.

Why a scalar is not enough. From one number per object, “*A is behind B*” is expressible. “*The third-closest person*” requires a total order over a set; “*between A and B in depth*” requires an interval test over three patches. Neither is available from a per-patch scalar without the program implementing the ordering itself — which the released prompt discourages, since it directs spatial relations to base Python on image coordinates.

5.1 C3 result and problem

On the RefCOCO+ depth/spatial subset:

4.26% $\rightarrow$ **19.15%**

However, non-spatial accuracy falls:

45.97% $\rightarrow$ **36.02%** (RefCOCO), 33.55% $\rightarrow$ **28.92%** (RefCOCO+).

The API-adoption study found **119 depth_order** calls under C3, where C1 and C2 had **zero**. This supports the interpretation that the generator was reaching for the new capability indiscriminately.

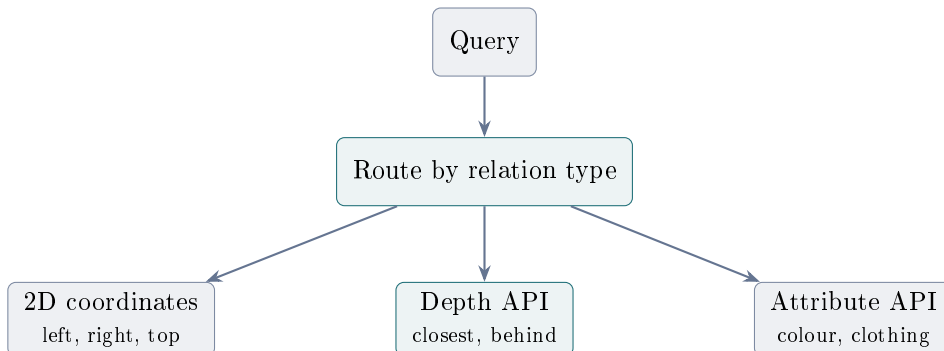
C3 lesson. “Giving the model a capability” and “telling the model when that capability is appropriate” are different problems. C3 solves the first and exposes the second.

6 C4 — depth capability plus routing rule

C4 = C3 + conditional routing specification

C4 keeps the same depth primitives. The new part is a prompt rule stating that depth functions should be used *only* when the query actually expresses a depth relation — *closest*, *nearest*, *farthest*, *behind*, *in front of*, *between in depth*. The prompt also provides examples of queries that should *not* use depth. C4 therefore teaches both:

WHEN to use depth + **WHEN NOT to use depth**



6.1 C4 results

Dataset / subset	C1	C2	C3	C4
RefCOCO+ overall	0.00	30.80	28.00	37.60
RefCOCO+ spatial, $n = 47$	0.00	4.26	19.15	21.28
RefCOCO+ non-spatial	0.00	33.55	28.92	39.29
RefCOCO overall	0.00	39.20	36.40	42.00
RefCOCO spatial, $n = 289$	0.00	34.26	36.68	37.02
RefCOCO non-spatial, $n = 211$	0.00	45.97	36.02	48.82

These are accuracy at $\text{IoU} \geq 0.5$ for the Qwen2.5-Coder-7B greedy experiment, with $n = 500$ per cell.

Main C4 finding. C4 retains the depth gain while recovering the non-spatial control performance. RefCOCO overall rises from 39.20% in C2 to 42.00% in C4; the RefCOCO+ depth/spatial subset rises from 4.26% to 21.28%. **C4 improves every cell of both datasets relative to C2** — it is a strict improvement rather than a trade.

6.2 Robustness of the C4 result

Greedy decoding is deterministic, so variance was estimated separately by sampling at $T = 0.7$ with three seeds on RefCOCO:

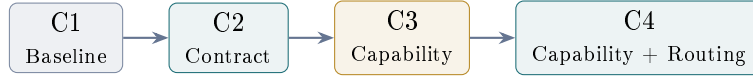
Condition	overall	spatial	non-spatial
C2 contract	36.07 ± 1.75	32.87 ± 3.35	40.44 ± 0.72
C3 + depth	34.40 ± 0.40	34.02 ± 0.80	34.92 ± 0.55
C4 + routing	42.07 ± 0.61	38.29 ± 0.72	47.23 ± 1.19

Two points follow. First, C4 separates from C2 by 6.0 points against standard deviations below two, so the result is not a decoding artefact; note also that C2’s greedy figure (39.20) sits 1.8 standard deviations *above* its own sampled mean, so the greedy table understates the difference.

Second, and unanticipated: **the primitives buy stability as well as accuracy**. Spatial-query standard deviation falls from ± 3.35 under C2 to ± 0.72 under C4, a factor of 4.6. Under C2 the generator improvises spatial logic afresh on every sample — sometimes sorting by coordinate, sometimes dropping the relation entirely — and which it does depends on the seed. Once **depth_order** exists there is one way to express the relation, and the variance collapses.

7 The complete C1–C4 table

	C1	C2	C3	C4
Released base prompt	Yes	Yes	Yes	Yes
Grounding return contract	No	Yes	Yes	Yes
Must return one ImagePatch	No	Yes	Yes	Yes
Spatial relation preservation	Implicit	Explicit	Explicit	Explicit
Explicit depth primitives	No	No	Yes	Yes
Depth routing rule	No	No	No	Yes
Negative “do not use depth” examples	No	No	No	Yes



8 Spatial versus non-spatial — how to explain it

The study separates queries into subsets using a relational-term lexicon. Spatial language includes image-plane relations such as *left*, *right*, *above* and *below*, and depth-related relations such as *closest*, *farthest*, *behind* and *in front*. The depth subset is the smaller set requiring depth ordering or 3D relational reasoning.

Non-spatial queries act as **controls**. They matter because if a new depth capability improves only the target subset while harming other queries, the change is a trade-off rather than an overall improvement — which is exactly what C3 turned out to be.

Why the two datasets differ. RefCOCO permits location words; RefCOCO+ forbids them. A manual audit of all 47 RefCOCO+ spatial queries showed that forbidding location words removed viewer-frame *directional* expressions but left *depth* ones. So RefCOCO’s spatial subset is predominantly 2D (“*man on right*”) and RefCOCO+’s is predominantly depth-based (“*man closest to us*”). That is why they score 34.26 and 4.26 respectively under C2, and why the depth primitives help one far more than the other.

The strongest single piece of evidence. Across three generator sizes — 1.5B, 7B and 32B — non-spatial accuracy rises $33.65 \rightarrow 45.97 \rightarrow 48.82$, while spatial accuracy stays flat at 34.95/34.26/34.95, on identical images with identical perception modules. A larger generator writes better programs for everything *except* spatial relations. If the spatial deficit were a program-synthesis problem, capacity would help; it demonstrably helps elsewhere. The bottleneck is therefore the interface, and that is what motivates C3 and C4.

9 Why the C1–C4 design is a controlled experiment

Each condition answers a separate question:

-
- C1** What happens with the released prompt?
 - C2** Does an explicit grounding output contract fix the grounding failure?
 - C3** Does explicit depth capability improve depth reasoning?
 - C4** Does routing the capability prevent overuse and improve the whole system?
-

This is stronger than changing the entire prompt at once, because it isolates the effect of each intervention. Each condition is a strict superset of the previous one; the model, the vision modules, the data and the decoding are held fixed throughout, and the execution stage re-runs *stored* programs, so a difference in accuracy can only come from generation.

10 What changed from the original ViperGPT paper?

Component	Original / reported setup	This reproduction
Generator	Codex <code>code-davinci-002</code>	Qwen2.5-Coder
Prompt study	Single released specification	C1–C4 controlled conditions
Pipeline	Query $\rightarrow$ program $\rightarrow$ execute	Same core method; generation and execution split by JSONL
Depth API	<code>compute_depth()</code> scalar	Explicit depth-ordering primitives in C3 / C4
Depth backend	MiDaS output	Sign corrected and validated
Verification	X-VLM	CLIP ViT-L substitution
Environment	Older published environment	Modernised for H100 execution
Threshold	Paper setup	Re-swept and validated at 0.2 for detection
Scoring	Paper reporting	Accuracy at $\text{IoU} \geq 0.5$ plus mean IoU; missing predictions score 0

Important claim boundary. Do *not* call 42.00% a faithful numerical reproduction of the original Codex result. Codex is retired and several implementation substitutions are documented. The defensible claim is that the ViperGPT *method* and the prompt-specification questions are reproduced under a documented modern implementation.

On the metric. The paper’s Table 1 is captioned “we report accuracy” while its column header reads “IoU (%)”, and the released evaluator computes mean IoU. This study reports accuracy at $\text{IoU} \geq 0.5$ as primary and mean IoU alongside; on these runs the two agree to about a point (42.00 against 41.81 for C4 on RefCOCO). Separately, the released evaluator substitutes the dataset mean box (worth ≈ 22.6 IoU) for a missing prediction, whereas this study scores an invalid return as zero. Under the paper’s own evaluator, C1 would read ≈ 22.6 rather than 0.00 — the programs are equally invalid either way; what differs is whether the evaluator conceals the failure.

11 What is the actual research contribution?

1. **Grounding specification gap.** The released prompt does not explicitly enforce the required `ImagePatch` return type. Adding the contract changes RefCOCO accuracy from 0.00% to 39.20%, and the effect is scale-independent across 1.5B, 7B and 32B generators.
2. **Depth reasoning gap.** After the grounding contract is fixed, depth-related queries remain weak. Explicit depth primitives raise the RefCOCO+ depth/spatial subset from 4.26% to 19.15% under C3 and to 21.28% under C4.
3. **Capability routing.** C3 demonstrates overuse of a new capability; C4 adds a routing rule and improves *every* reported RefCOCO and RefCOCO+ subset relative to C2, reaching 42.00% overall and reducing spatial-query variance by $4.6\times$.
4. **Methodological discipline.** Two results large enough to be attractive were rejected after control experiments. Capping the detector’s candidate set to three raises RefCOCO accuracy from 42.80% to 57.60% — but applying the same cap to C2, which contains *no* depth primitives, yields almost the same gain (+22.5 against +23.9 points on the spatial subset). The

improvement therefore belongs to the candidate set, not to the method, and is not reported as a result.

12 Summary

“My experiment has four controlled prompt conditions. C1 is the released ViperGPT prompt and acts as the baseline. It does not explicitly specify that a grounding program must return an ImagePatch, so many generated programs return strings and grounding accuracy is 0%.

In C2 I add only a grounding contract: the function must return exactly one ImagePatch and must preserve spatial relations. With the same model and data, RefCOCO rises from 0% to 39.2%.

In C3 I address depth reasoning by adding explicit primitives such as `depth_order`, `is_behind` and `is_in_front_of`. Depth-query performance improves strongly, but non-spatial accuracy falls because the generator overuses the new capability.

Therefore C4 keeps the same depth primitives but adds a routing rule. The prompt says to use depth functions only for actual depth relations, and gives examples where depth should not be used. C4 reaches 42.0% overall on RefCOCO and 21.28% on the RefCOCO+ depth/spatial subset while recovering the non-spatial controls.

So the progression is: C1 identifies the specification gap, C2 fixes the output contract, C3 adds the missing capability, and C4 controls when that capability is used.”

13 Questions and answers

Why C1–C4? To isolate the effect of each intervention. Changing everything at once would not show which change caused the improvement.

Difference C1 versus C2? C1 leaves the grounding output type implicit; C2 explicitly requires one ImagePatch and preservation of spatial relations.

Difference C2 versus C3? C2 fixes the output interface; C3 adds explicit depth reasoning capability.

Difference C3 versus C4? C3 provides depth tools; C4 provides the same tools plus a rule for when they should and should not be used.

Why does C3 hurt non-spatial accuracy? The generator overuses the newly exposed depth capability on queries that should use 2D coordinates or attribute verification. `depth_order` was called 119 times on RefCOCO, where most queries need no depth at all.

Why is C4 better? It preserves the depth capability while constraining its use, so the target subset improves without the collateral loss — and every other subset improves as well.

Is the Qwen replacement your contribution? No. It is a necessary present-day reproduction substitution because Codex is retired. The research contribution is the controlled diagnosis of specification gaps and capability routing.

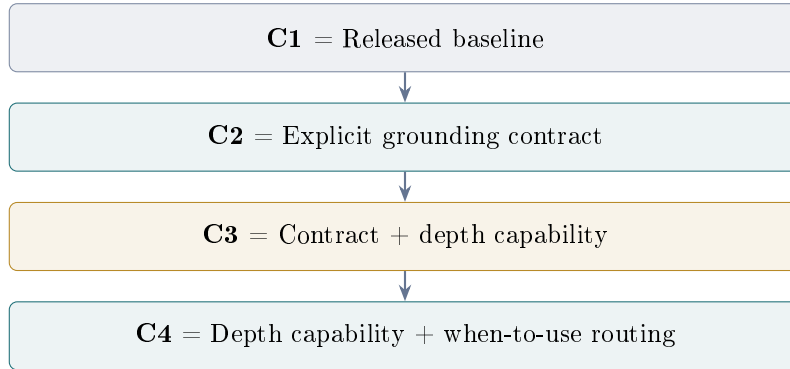
Did C1 really fail? It failed as a grounding interface. Many programs executed successfully and returned strings, but a string has no predicted box and therefore cannot receive an IoU score. This is why the failure is described as a specification failure rather than a crash.

Is the improvement just the smaller candidate set? No, and this was tested directly. Capping the detector’s candidates does raise accuracy substantially, but a control condition without any depth primitives gains almost as much, so that gain is attributable to the candidate set rather than to the method. It is therefore disabled in all reported results.

How do you know the depth signal is correct? It was validated against ground-truth boxes with the detector removed entirely. With the corrected sign the annotated target lands at the correct depth extreme in 20 of 25 cases; without it, in 2 of 25, against a chance level of 26%. A score *below* chance indicates an inverted signal rather than a weak one.

What is your strongest conclusion? Capability and specification are separate concerns. A model needs both the capability and a clear rule for when that capability applies.

14 Final



C1 identifies the problem → C2 fixes the contract → C3 adds capability → C4 controls when it is used.

Final. The key result is not merely that C4 has a higher number. The controlled progression shows *why*: C2 fixes an output-interface omission, C3 adds a missing depth capability, and C4 demonstrates that the capability must be accompanied by an explicit condition describing when it applies.

All figures reported here are traceable to stored run summaries and scheduler job identifiers. The C2 return-count figures (336 string returns, 443 patch returns) and the RefCOCO+ C2 accuracy of 31.80% come from the Milestone 2 execution set; the C1–C4 comparison table in Section 7 comes from the Milestone 3 set, in which the corresponding values are 332, 444 and 30.80%. The two sets differ only in run provenance and not in any conclusion drawn.